

INTERNSHIP REPORT

on

PYTHON PROGRAMMING

Submitted in partial fulfilment of the requirements for the degree of

**Bachelor of Technology In
Computer Science & Engineering (Data Science)**



Government Engineering College, Lakhisarai

Under the supervision of



Md. Azad

(UpEducative Technology Private Limited)

Name	Isha Gupta
Registration No.	25153158029
Branch	Computer Science & Engineering (Data Science)
Session	2026–2027
College	Government Engineering College, Lakhisarai

DECLARATION

I, Isha Gupta, a student of Bachelor of Technology in Computer Science and Engineering (Data Science) at Government Engineering College, Lakhisarai, hereby declare that this internship report entitled “Python Programming” is my original work, based on the knowledge and practical experience gained during my internship at UpEducative Technology Private Limited.

I have prepared this report as a part of my academic and internship requirements. The information, concepts, practical programs and learning outcomes presented in this report are based on the training and hands-on exercises undertaken during the internship, under the supervision of my mentor, Md. Azad.

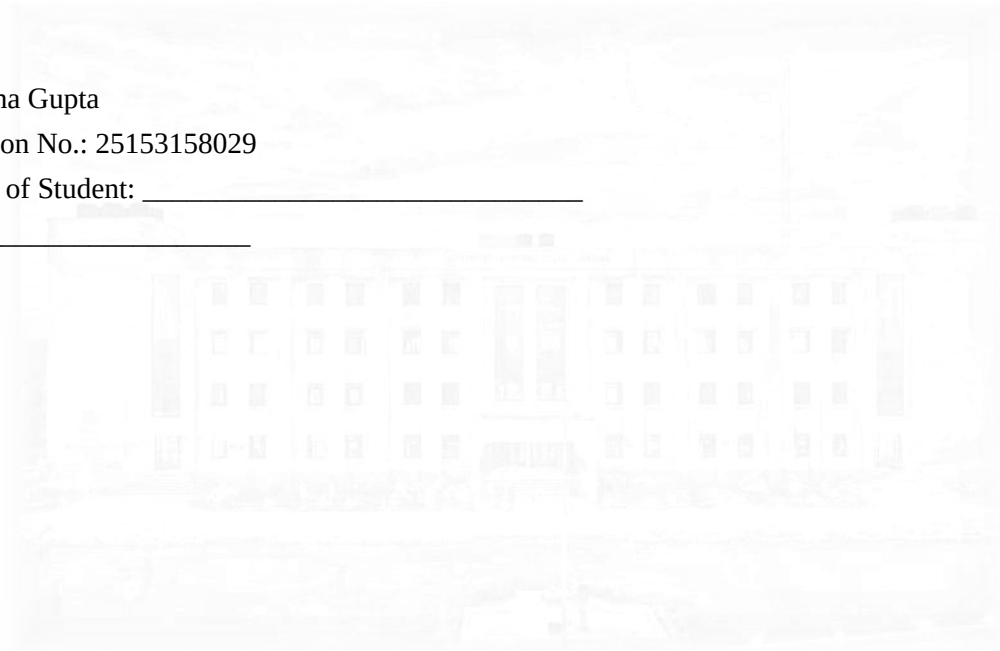
I further declare that this report has not been submitted, either wholly or partially, to any other institution or organization for the award of any degree, diploma or certificate, and that it has been prepared independently in my own words.

Name: Isha Gupta

Registration No.: 25153158029

Signature of Student: _____

Date: _____



ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Government Engineering College, Lakhisarai for providing me with the opportunity to undertake the Python Programming internship at UpEducative Technology Private Limited and gain valuable practical knowledge and technical skills.

I am deeply thankful to my internship mentor, Md. Azad, for his continuous guidance, support and encouragement throughout the internship. His valuable suggestions and supervision helped me understand the concepts of Python programming and Data Structures and apply them through five practical mini projects — the ATM Simulation, Car Drive Simulation, Login System, Student Marks Analyzer and Number Analyzer.

This internship enabled me to strengthen my understanding of Python fundamentals, including variables, data types, operators, conditional statements, loops, functions, lists, dictionaries, file handling and exception handling, along with basic Data Structures and Algorithms concepts such as traversal, searching and counting.

Finally, I am grateful to my faculty members and everyone who directly or indirectly contributed to the successful completion of my internship and the preparation of this report.

Isha Gupta

B.Tech CSE (Data Science)

Government Engineering College, Lakhisarai

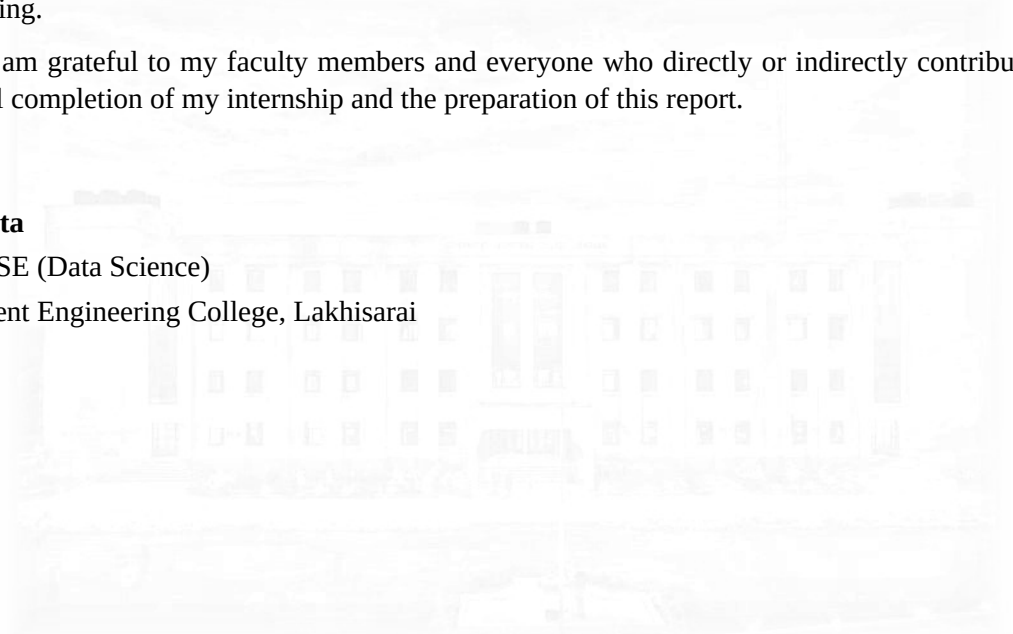


TABLE OF CONTENT

01	Introduction
02	Basics of Python
03	Variables, Data Types & Operators
04	Control Flow Statements
05	Functions in Python
06	Data Structures Used (Lists & Dictionaries)
07	Input Validation, Exception Handling & File Handling
08	Basic DSA Concepts
09	Practical Programs
10	Tools and Technologies Used
11	Learning Outcomes
12	Conclusion



SECTION 1

1. Introduction

Python is a high-level, interpreted, general-purpose programming language widely used for developing different types of software applications. It is known for its simple and readable syntax, which makes it suitable for both beginners and professional developers, and it supports multiple programming paradigms including procedural, object-oriented and functional programming.

As part of the Python Programming internship at UpEducative Technology Private Limited, various fundamental concepts of Python and basic Data Structures and Algorithms (DSA) were studied and implemented through five practical mini projects: an ATM Simulation, a Car Drive Simulation, a Login System, a Student Marks Analyzer and a Number Analyzer.

The purpose of building these projects was not just to write working code, but to understand how different programming concepts — variables, data types, conditions, loops, functions, lists, dictionaries, file handling and exception handling — come together to form a complete, working program. The topics covered during the internship included variables, data types, operators, input and output, type conversion, conditional statements, loops, functions, lists, dictionaries, input validation, exception handling, menu-driven program design, file handling, JSON, the datetime module, and basic DSA concepts such as traversal, searching, and counting.

Each of the five projects was chosen to exercise a different combination of these ideas. The ATM Simulation and Car Drive Simulation are both menu-driven programs built around a central data structure (a dictionary) that is updated on every user action; the Login System focuses on validation and controlled access; the Student Marks Analyzer and Number Analyzer focus on processing a collection of values — a list, or a list of dictionaries — using functions, loops and simple DSA techniques such as traversal and searching. Studying them together, rather than as isolated exercises, is what ties the theory in Sections 2–8 to the actual, working code in Section 9.

About the Company

UpEducative Technology Private Limited is the ed-tech and training partner through which this Python Programming internship was conducted. The company runs structured, project-based internship training programmes for engineering students, pairing a dedicated mentor with each intern and organising the training around building real, working projects rather than passive lectures — the same approach that shaped the five mini projects documented in this report, carried out under the supervision of mentor Md. Azad.

Note: This report first builds up the Python and DSA theory used across the internship (Sections 2–8), and then walks through each of the five mini projects individually (Section 9), showing how that same theory was applied in real, working code.

SECTION 2

2. Basics of Python

Before understanding the five mini projects built during this internship, it is important to understand some basic Python concepts that are used throughout them. Python programs are read and executed from top to bottom, and unlike many other languages, Python does not use curly braces { } to mark a block of code — it uses indentation (consistent spacing) instead. This is why every function, loop and if-block in the source code shown later in this report is indented carefully; incorrect indentation in Python is not just a style issue, it is a syntax error.

Python Program Flow

A Python program is made up of instructions that tell the computer what to do, executed one after another. Every program built during this internship follows the same basic flow, repeated over and over inside a loop for as long as the program's menu is running:

User gives input → Program processes the input → Program gives output

```
name = input("Enter your name: ")
print("Hello", name)
```

Variables

A variable is a name used to store a value in a program so that it can be reused or updated later. Programs need to remember information while they are running — for example, in the Car Drive Simulation, the program needs to remember the current speed and fuel between one menu choice and the next, so these values are stored in variables rather than being recalculated from scratch every time:

```
speed = 50
fuel = 20
```

A variable's value can change while the program runs — this is exactly what happens every time the ATM Simulation updates data["balance"] after a deposit, or the Car Drive Simulation updates car["speed"] after acceleration. Python is also dynamically typed, meaning the same variable name can be reassigned a value of a different type later, though the projects in this report keep each variable's type consistent throughout.

Data Types

A data type tells Python what kind of value is being worked with, and determines which operations are valid on it — for instance, two strings can be joined with +, but a string and a number cannot be added directly without first converting one of them. The main data types used across these projects are:

- String — used for text, such as student names, usernames, passwords and account holder names.
Example: name = "Isha"
- Integer — used for whole numbers, such as a count of login attempts or the number of students.
Example: attempts = 3
- Float — used for numbers with decimal values, such as marks, money and fuel. Example: balance = 2500.50

- Boolean — has only two values, True or False, and is normally used as a flag or a status. In the Car Drive Simulation, `engine = False` means the engine is OFF, and `engine = True` means it is ON; the whole `accelerate()` and `drive()` functions check this single Boolean before doing anything else.

Input, Type Conversion and Output

The `input()` function is used whenever the program needs the user to type something in. By default, `input()` always returns the entered value as a string — even if the user types a number like 85, Python initially treats it as the text "85", not the number 85. To use it as a number for calculations or comparisons, type conversion is required:

```
marks = int(input("Enter marks: "))
amount = float(input("Enter amount: "))
```

Here, `int()` converts the entered text into a whole number and `float()` converts it into a decimal number; `str()` is used in the opposite direction, to convert a value into a string when it needs to be joined with other text. The `print()` function is then used to display information on the screen — menus, results, error messages, balances, marks and car status are all shown using `print()`.

Why this matters: `int()` and `float()` raise a `ValueError` if the text entered cannot be converted — for example, if a user types “abc” instead of a number. Every project in this report that asks for a number wraps this conversion inside a `try-except` block (covered in Section 7) so the program can show a friendly message instead of crashing.

SECTION 3

3. Variables, Data Types & Operators

Operators are symbols used to perform operations on values, and every one of the five mini projects relies on them for its calculations and decisions. The table below lists the operators actually used across the projects — for example, the Car Drive Simulation calculates $\text{Distance} = \text{Speed} \times \text{Time}$ using `*`, and the ATM Simulation uses `+=` and `-=` to update the balance after every deposit and withdrawal rather than writing `balance = balance + deposit` in full.

Operator	Meaning	Example
<code>+</code>	Addition	<code>10 + 5</code>
<code>-</code>	Subtraction	<code>10 - 5</code>
<code>*</code>	Multiplication	<code>10 * 5</code>
<code>/</code>	Division	<code>10 / 5</code>
<code>%</code>	Remainder (Modulus)	<code>10 % 3</code>
<code>==</code>	Equal to (comparison)	<code>marks == 100</code>
<code>>=</code>	Greater than or equal to	<code>marks >= 50</code>
<code>+=</code>	Add and assign	<code>balance += deposit</code>
<code>-=</code>	Subtract and assign	<code>balance -= withdraw</code>

The Modulus Operator

The `%` operator gives the remainder left over after division, rather than the result of the division itself — for example, `10 % 3` is 1, because 3 goes into 10 three times with 1 left over. This is especially useful for checking whether a number is even or odd, since dividing any even number by 2 always leaves a remainder of 0. This exact technique is used directly in the Number Analyzer to count even and odd values:

```
number % 2 == 0    # True means the number is even
```

For example, `10 % 2 = 0` (even) while `15 % 2 = 1` (odd). The same operator idea, applied to 4-digit PIN and password checks, is also why the ATM Simulation can rely on simple comparisons rather than more complex validation logic.

SECTION 4

4. Control Flow Statements

By default, Python runs every line of a program in order, from top to bottom. Control flow statements — conditions and loops — are what let a program skip lines, repeat lines, or choose between different paths, and they are the backbone of every menu-driven project in this report.

Conditions (if, elif, else)

Conditions allow a program to make decisions. When there are multiple possibilities, if, elif and else are used together — Python checks each condition from top to bottom and runs the block under the first one that turns out to be true, skipping all the rest. This is exactly how the Student Marks Analyzer assigns a grade:

```
if marks >= 90:
    grade = "A+"
elif marks >= 80:
    grade = "A"
elif marks >= 70:
    grade = "B"
else:
    grade = "F"
```

The order of the conditions matters here — because 95 also satisfies marks ≥ 80 , the check for marks ≥ 90 has to come first, or every A+ student would incorrectly be graded A instead. Conditions are used throughout all five projects: checking whether the ATM balance is sufficient before a withdrawal, whether the car's engine is ON before it can accelerate, whether marks fall inside the valid 0–100 range, and whether the entered login details match the stored ones.

Loops

A loop is used to repeat a block of instructions without writing it out multiple times. Two kinds of loops are used across the projects, and the choice between them depends on whether the number of repetitions is known in advance:

- for loop — used when a known collection of items needs to be processed one by one, such as looping through every student in the Student Marks Analyzer or every number in the Number Analyzer.
- while loop — used when a program must keep running for as long as a condition stays true, and the number of repetitions is not known ahead of time — such as the ATM and Car menus (while True, which repeats forever until the user explicitly exits), or asking the user for valid input again and again, or giving the user a limited number of login attempts.

```
numbers = [10, 20, 30]

for number in numbers:
    print(number)

attempts = 3
```

```
while attempts > 0:  
    # login code  
    attempts -= 1
```

Note: A while loop that never has its condition turned false becomes an infinite loop. Every while True menu in this report is paired with a break statement (triggered by an “Exit” or “Logout” choice) so the loop always has a way out.



SECTION 5

5. Functions in Python

A function is a named, reusable block of code created for a particular task. Functions are useful because a large program can be divided into smaller, self-contained parts that are easier to write, read and fix — instead of writing the complete ATM program as one long block, different tasks are separated into functions such as `create_account()`, `login()`, `deposit_money()`, `withdraw_money()`, `check_balance()` and `change_pin()`, each responsible for exactly one job.

```
def find_largest(numbers):  
    largest = numbers[0]  
  
    for number in numbers:  
        if number > largest:  
            largest = number  
  
    return largest
```

Here, `def` is used to create the function, `find_largest` is the function name, and `numbers` is a parameter — a placeholder for whatever information the function will receive from outside when it is called. When the function is actually called as `find_largest(my_numbers)`, `my_numbers` is the argument being passed in to fill that placeholder.

A function can send a result back to the code that called it using `return`. In the example above, the largest value found after checking every number is returned, and can then be stored in a variable, such as `result = find_largest(numbers)`. This pattern of parameters, arguments and return values is used extensively in the Number Analyzer, where separate functions calculate the sum, subtraction, largest and smallest values, and each one simply returns its answer rather than printing it directly — which keeps the calculation logic separate from the display logic, and is considered good programming practice.

SECTION 6

6. Data Structures Used (Lists & Dictionaries)

Lists

A list is used to store multiple values together in a single, ordered collection, instead of creating many separate variables. Lists are used across the projects for storing numbers, students and transactions — anywhere the program needs to keep track of “all of these” rather than just one value:

```
numbers = [10, 20, 30, 40]
```

Each item in a list has a position called an index, and Python starts indexing from 0 — so `numbers[0]` gives 10, not 20. The `append()` function adds a new item to the end of a list without disturbing the items already there; in the ATM project it is used to add new transactions to the transaction list every time a deposit or withdrawal happens, and in the Student Marks Analyzer it is used to add each student's record to the student list one at a time as they are entered. The `len()` function tells how many items a list currently contains — used, for example, to calculate `Average = Total Marks / len(students)` without needing to separately keep count of how many students were entered.

Dictionaries

A dictionary stores information using key–value pairs, which is useful whenever several different pieces of information all belong to the same real-world object — for example, a single student has a name, marks and a grade, and a dictionary lets all three be stored and accessed together instead of in three unrelated variables:

```
student = {  
    "name": "Isha",  
    "marks": 85,  
    "grade": "A"  
}
```

A dictionary value is accessed using its key inside square brackets, such as `student["marks"]` or, in the ATM project, `data["balance"]`. Dictionaries are used for ATM account information (holder name, account number, PIN, balance, transactions), for the car's entire state (speed, distance, fuel, engine), and for login information (username and password) — in each case because the program is really modelling one object with several related properties.

List of Dictionaries

When many objects each need to store several different pieces of information, dictionaries are combined with lists — exactly how the Student Marks Analyzer stores its whole class, with each individual student represented as one dictionary, and every student's dictionary collected together inside a single list:

```
students = [  
    {"name": "Isha", "marks": 85, "grade": "A"},  
    {"name": "Rahul", "marks": 72, "grade": "B"}  
]
```

This list-of-dictionaries pattern is what makes it possible to later loop through students (a for loop, Section 4), read each one's marks (student["marks"], this section), and compare them against a running highest score (searching, Section 8) — all three ideas combined in one small piece of code.



SECTION 7

7. Input Validation, Exception Handling & File Handling

Input Validation

Input validation means checking whether the information entered by the user is acceptable before the program acts on it, since users can and do enter incorrect, incomplete or out-of-range data. Rather than trusting every input blindly, each project re-asks for input, or refuses the action, until a valid value is given:

- ATM — PIN must contain exactly 4 digits; deposit and withdrawal amounts cannot be negative; withdrawal cannot exceed the balance.
- Student Marks Analyzer — marks must be between 0 and 100; student name cannot be empty.
- Login System — username cannot be empty; password must contain at least 4 characters.
- Car Drive Simulation — acceleration and fuel amounts must be positive; speed cannot go beyond the maximum limit.

try-except (Exception Handling)

Even with validation in place, an unexpected input can still cause a program to crash with an error — for example, `float(input(...))` raises a `ValueError` if the user types text such as “abc” instead of a number. `try-except` is used to catch this kind of error and show a proper, friendly message instead of letting the whole program stop suddenly and unhelpfully:

```
try:
    amount = float(input("Enter amount: "))
except ValueError:
    print("Please enter a valid amount.")
```

Python attempts to run everything inside the `try` block first; if a `ValueError` occurs at any point inside it, control jumps straight to the `except` block instead of crashing the program. This pattern is used in the ATM, Car Drive Simulation, Number Analyzer and Student Marks Analyzer for every piece of numeric input they take.

Menu-Driven Programming

A menu-driven program gives the user multiple choices, printed as a numbered list, and then checks the choice using `if-elif-else` to call the matching function. This structure is used in the ATM Simulation and Car Drive Simulation, where a menu is displayed repeatedly inside a `while` loop until the user deliberately chooses to exit — combining loops, conditions and functions into one coherent piece of program design.

File Handling, JSON and datetime

Normally, a program's variables lose their values the moment it closes — every deposit, withdrawal and PIN change would be forgotten the next time the ATM program is run. To keep data available across separate runs, the ATM Simulation saves it to a file named `atm_data.json` using Python's built-in `json` module, which reads existing data at startup, updates it in memory as the user interacts with the menu, and saves the whole dictionary back to disk after every change:

```
def save_data(data):  
    with open(DATA_FILE, "w") as file:  
        json.dump(data, file, indent=4)
```

The ATM project also uses Python's datetime module to record the exact date and time of every transaction, which is what makes the mini statement and last-transaction features meaningful rather than just a bare list of numbers:

```
"date": datetime.now().strftime("%d-%m-%Y %H:%M:%S")
```

Note: `json.dump()` can only save data types it understands — dictionaries, lists, strings, numbers and Booleans. This is why the ATM's account dictionary (holder name, PIN, balance, and a list of transaction dictionaries) saves to JSON cleanly: every value inside it is one of these basic types.



SECTION 8

8. Basic DSA Concepts

Although these are beginner-level projects, they also apply some fundamental Data Structures and Algorithms (DSA) ideas — the same ideas that scale up to much larger, real-world systems. A data structure is simply a way of organising and storing data so that it can be used efficiently; here, the two data structures used are lists (for an ordered collection of values) and dictionaries (for one object's related properties), as covered in Section 6.

Traversal

Traversal means visiting every element of a list one by one, in order, usually to read, check or accumulate something from each one. It is the single most common operation in these projects, forming the basis of the Number Analyzer's sum and average calculations, the Student Marks Analyzer's loop through every student, and the ATM's processing of its transaction list for the mini statement.

Searching — Finding the Largest and Smallest

The Number Analyzer finds the largest number using a simple linear search: it starts by assuming the first number is the largest, then compares every other number against that assumption, replacing it whenever a bigger value turns up. For example, for [10, 25, 15, 40, 20]: start with 10 as the largest; 25 is bigger, so largest becomes 25; 15 is smaller, so 25 is kept; 40 is bigger, so largest becomes 40; 20 is smaller — the final largest is 40. The same approach, with the comparison reversed, is used to find the smallest number, and this exact technique reappears in the Student Marks Analyzer to find the class's highest scorer.

Counting

Counting means keeping a running tally of how many times something happens while traversing a list. The Number Analyzer counts even and odd numbers by starting two counters, `even_count` and `odd_count`, at 0, and increasing the correct one — using the modulus check from Section 3 — as every number in the list is visited.

Why this matters: Traversal, searching and counting look simple here because the lists involved are small, but they are exactly the same building blocks used inside far larger real-world systems — the same “compare against the current best” idea used to find the largest number is, at its core, how a search engine or a leaderboard finds a top result.

SECTION 9

9. Practical Programs

This section presents all five mini projects built during the internship in full detail — the objective and problem statement behind each one, the concepts it uses (linking back to Sections 2–8), the algorithm it follows, its complete, syntax-highlighted source code, and its sample input and output, followed by the specific challenges faced while building it.

How to read this section: Each project follows the same eight-part structure — Objective, Problem Statement, Concepts Used, Program Working, Algorithm, Source Code, Sample Output, and Errors/Challenges Faced — so that the reasoning behind the code can be followed before looking at the code itself.



PROJECT 1

ATM Simulation

Program File: `atm_simulation.py`

I. Objective

To develop a menu-driven ATM simulation program that lets a user create an account, log in securely with a 4-digit PIN, and perform banking operations such as checking balance, depositing, withdrawing, viewing transaction history and changing the PIN — with all account data stored permanently in a JSON file.

II. Problem Statement

The program must let a user set up an account with a name, a 4-digit PIN and an initial deposit; verify the PIN at login with a maximum of 3 attempts; allow deposits and withdrawals (rejecting withdrawals greater than the balance); show the last five transactions and the most recent one; allow the PIN to be changed only after the current PIN is verified; and save every change to a JSON file so the account is remembered the next time the program runs.

III. Concepts Used

- Functions — the whole program is broken into functions such as `create_account()`, `login()`, `deposit_money()`, `withdraw_money()`, `mini_statement()`, `change_pin()` and `account_details()`.
- Dictionaries — account information (holder name, account number, PIN, balance) is stored as key–value pairs.
- Lists — every deposit and withdrawal is appended to a transactions list.
- File Handling & JSON — the account dictionary is saved to and loaded from `atm_data.json` using the `json` module, so data persists between runs.
- `datetime` module — every transaction is timestamped using `datetime.now()`.
- `while` loop — used for the main menu (`while True`) and for repeating PIN validation until it is correct.
- `if-elif-else` — used to route the user's menu choice to the correct function.
- `try-except` — used to catch `ValueError` when the deposit or withdrawal amount is not a valid number.
- Input validation — checking PIN length (`isdigit()` and length 4), and that deposit/withdrawal amounts are positive.

IV. Program Working

- When the program starts, it loads existing data from `atm_data.json`; if no account exists yet, it asks the user to create one.
- Account creation asks for the holder's name, a 4-digit PIN (with confirmation) and an initial deposit, then generates an account number and saves everything to the JSON file.
- At login, the user has 3 attempts to enter the correct PIN; after 3 failed attempts, access is blocked for that session.

- Once logged in, a menu is displayed repeatedly, letting the user check the balance, deposit, withdraw, view the mini statement or last transaction, change the PIN, view account details, or log out.
- Every deposit or withdrawal updates the balance, appends a timestamped transaction, and immediately saves the updated data back to the JSON file.

Why this matters: *The ATM Simulation is the only one of the five projects whose data survives after the program closes — every other project starts fresh each run. This is entirely because of the `load_data()` / `save_data()` pair built on Python's `json` module, covered in Section 7.*

V. Algorithm / Steps

1. Load existing account data from the JSON file (or start with empty/default values).
2. If no account exists, create one — take the holder's name, a validated 4-digit PIN, and an initial deposit, then save the data.
3. Ask for the PIN and compare it with the stored PIN, allowing up to 3 attempts.
4. If the PIN is correct, display the main menu inside a loop.
5. Take the user's menu choice and call the matching function (balance, deposit, withdraw, mini statement, last transaction, change PIN, or account details).
6. For deposits, add the amount to the balance; for withdrawals, first check that the balance is sufficient before subtracting.
7. Record every deposit/withdrawal as a transaction with its type, amount and date/time.
8. Save the updated data to the JSON file after every change.
9. Repeat the menu until the user chooses to log out.

VI. Source Code

```
import json
import os
from datetime import datetime

DATA_FILE = "atm_data.json"

# ----- FILE HANDLING -----

def load_data():
    if os.path.exists(DATA_FILE):
        with open(DATA_FILE, "r") as file:
            return json.load(file)

    return {
        "account_holder": "",
        "account_number": "",
        "pin": "",
        "balance": 0,
        "transactions": []
    }
```

```
}

def save_data(data):
    with open(DATA_FILE, "w") as file:
        json.dump(data, file, indent=4)

# ----- ACCOUNT SETUP -----

def create_account(data):
    print("\n===== ATM ACCOUNT SETUP =====")

    name = input("Enter account holder name: ").strip()

    while True:
        pin = input("Create a 4-digit PIN: ")

        if pin.isdigit() and len(pin) == 4:
            confirm_pin = input("Confirm your PIN: ")

            if pin == confirm_pin:
                break
            else:
                print("PINs do not match. Try again.")
        else:
            print("PIN must contain exactly 4 digits.")

    while True:
        initial_deposit = input("Enter initial deposit amount: ")

        try:
            initial_deposit = float(initial_deposit)

            if initial_deposit >= 0:
                break
            else:
                print("Amount cannot be negative.")

        except ValueError:
            print("Please enter a valid amount.")

    account_number = "ATM" + str(datetime.now().timestamp()).replace(".", "")[-8:]

    data["account_holder"] = name
    data["account_number"] = account_number
    data["pin"] = pin
    data["balance"] = initial_deposit

    if initial_deposit > 0:
```

```
        data["transactions"].append({
            "type": "Initial Deposit",
            "amount": initial_deposit,
            "date": datetime.now().strftime("%d-%m-%Y %H:%M:%S")
        })

    save_data(data)

    print("\nAccount created successfully!")
    print("Account Number:", account_number)

# ----- LOGIN -----

def login(data):
    print("\n===== ATM LOGIN =====")

    attempts = 3

    while attempts > 0:
        entered_pin = input("Enter your 4-digit PIN: ")

        if entered_pin == data["pin"]:
            print("\nLogin successful!")
            print("Welcome,", data["account_holder"])
            return True

        attempts -= 1

    if attempts > 0:
        print("Incorrect PIN.")
        print("Attempts remaining:", attempts)

    print("\nToo many incorrect attempts.")
    print("ATM access has been blocked for this session.")

    return False

# ----- CHECK BALANCE -----

def check_balance(data):
    print("\n===== BALANCE =====")
    print("Available Balance: Rs.", format(data["balance"], ".2f"))

# ----- DEPOSIT -----

def deposit_money(data):
    print("\n===== DEPOSIT =====")
```

```
try:
    amount = float(input("Enter amount to deposit: "))

    if amount <= 0:
        print("Invalid amount.")
        return

    data["balance"] += amount

    data["transactions"].append({
        "type": "Deposit",
        "amount": amount,
        "date": datetime.now().strftime("%d-%m-%Y %H:%M:%S")
    })

    save_data(data)

    print("Amount deposited successfully.")
    print("Updated Balance: Rs.", format(data["balance"], ".2f"))

except ValueError:
    print("Please enter a valid amount.")

# ----- WITHDRAW -----

def withdraw_money(data):
    print("\n===== WITHDRAW =====")

    try:
        amount = float(input("Enter amount to withdraw: "))

        if amount <= 0:
            print("Invalid amount.")

        elif amount > data["balance"]:
            print("Insufficient balance.")

        else:
            data["balance"] -= amount

            data["transactions"].append({
                "type": "Withdrawal",
                "amount": amount,
                "date": datetime.now().strftime("%d-%m-%Y %H:%M:%S")
            })

            save_data(data)
```

```
        print("Please collect your cash.")
        print("Remaining Balance: Rs.",
              format(data["balance"], ".2f"))

    except ValueError:
        print("Please enter a valid amount.")

# ----- MINI STATEMENT -----

def mini_statement(data):
    print("\n===== MINI STATEMENT =====")

    if len(data["transactions"]) == 0:
        print("No transactions available.")
        return

    for transaction in data["transactions"][-5:]:
        print(
            transaction["date"],
            "|",
            transaction["type"],
            "| Rs.",
            format(transaction["amount"], ".2f")
        )

    print("-----")
    print("Current Balance: Rs.",
          format(data["balance"], ".2f"))

# ----- LAST TRANSACTION -----

def last_transaction(data):
    print("\n===== LAST TRANSACTION =====")

    if len(data["transactions"]) == 0:
        print("No transactions found.")
        return

    transaction = data["transactions"][-1]

    print("Type:", transaction["type"])
    print("Amount: Rs.", format(transaction["amount"], ".2f"))
    print("Date:", transaction["date"])

# ----- CHANGE PIN -----

def change_pin(data):
```

```
print("\n===== CHANGE PIN =====")

current_pin = input("Enter current PIN: ")

if current_pin != data["pin"]:
    print("Incorrect current PIN.")
    return

while True:
    new_pin = input("Enter new 4-digit PIN: ")

    if not new_pin.isdigit() or len(new_pin) != 4:
        print("PIN must contain exactly 4 digits.")
        continue

    confirm_pin = input("Confirm new PIN: ")

    if new_pin != confirm_pin:
        print("PINs do not match.")
        continue

    data["pin"] = new_pin
    save_data(data)

    print("PIN changed successfully.")
    break

# ----- ACCOUNT DETAILS -----

def account_details(data):
    print("\n===== ACCOUNT DETAILS =====")

    print("Account Holder:", data["account_holder"])
    print("Account Number:", data["account_number"])
    print("Balance: Rs.", format(data["balance"], ".2f"))

# ----- ATM MENU -----

def atm_menu(data):

    while True:

        print("\n=====")
        print("          ATM MAIN MENU")
        print("=====")
        print("1. Check Balance")
        print("2. Deposit Money")
        print("3. Withdraw Money")
```

```
print("4. Mini Statement")
print("5. Last Transaction")
print("6. Change PIN")
print("7. Account Details")
print("8. Logout")
print("=====")

choice = input("Enter your choice: ")

if choice == "1":
    check_balance(data)

elif choice == "2":
    deposit_money(data)

elif choice == "3":
    withdraw_money(data)

elif choice == "4":
    mini_statement(data)

elif choice == "5":
    last_transaction(data)

elif choice == "6":
    change_pin(data)

elif choice == "7":
    account_details(data)

elif choice == "8":
    print("\nYou have been logged out.")
    print("Thank you for using the ATM.")
    break

else:
    print("Invalid choice. Please try again.")

# ----- MAIN PROGRAM -----

def main():

    data = load_data()

    if data["pin"] == "":
        create_account(data)

    if login(data):
        atm_menu(data)
```

```
main()
```

VII. Sample Input & Output

```
===== ATM LOGIN =====
Enter your 4-digit PIN:
Incorrect PIN.
Attempts remaining: 2
Enter your 4-digit PIN:
Incorrect PIN.
Attempts remaining: 1
Enter your 4-digit PIN:

Too many incorrect attempts.
ATM access has been blocked for this session.
PS C:\Users\ASUS\OneDrive\Desktop\DSA-Python-Mini-Projects> py atm_simulation.py
Could not find platform independent libraries <prefix>

===== ATM LOGIN =====
Enter your 4-digit PIN: 1234

Login successful!
Welcome, isha gupta

=====
                ATM MAIN MENU
=====
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Mini Statement
5. Last Transaction
6. Change PIN
7. Account Details
8. Logout
=====
Enter your choice: 3

===== WITHDRAW =====
Enter amount to withdraw: 44444
Please collect your cash.
Remaining Balance: Rs. 955556.00
```

VIII. Errors / Challenges Faced

- Generating a unique account number and storing the PIN, balance and transaction history safely in a JSON file between separate runs of the program.
- Making sure a withdrawal is never allowed to exceed the available balance.
- Formatting the balance to always display with exactly two decimal places using `format(value, ".2f")`.
- Handling non-numeric input for deposit/withdrawal amounts gracefully using `try-except` instead of letting the program crash.

PROJECT 2

Car Drive Simulation

Program File: *car_drive_simulation.py*

I. Objective

To simulate the basic working of a car — starting and stopping the engine, accelerating, braking, driving and refuelling — using a dictionary to keep track of the car's speed, distance travelled, fuel level and engine status.

II. Problem Statement

The program must not allow the car to accelerate or drive unless the engine is ON and fuel is available; must keep the speed within 0 to 120 km/h and the fuel within 0 to 50 litres; must not allow the engine to be switched off while the car is still moving; and must correctly calculate the distance travelled and the fuel consumed for every drive using $\text{Distance} = \text{Speed} \times \text{Time}$.

III. Concepts Used

- Dictionary — the car's state (speed, distance, fuel, engine) is stored together as one dictionary that is passed between functions.
- Functions — one function for each action: `start_engine()`, `stop_engine()`, `accelerate()`, `brake()`, `drive()`, `refuel()` and `show_status()`.
- while loop and if-elif-else — used for the menu-driven structure, identical in style to the ATM Simulation.
- Nested conditions — e.g. checking both that the engine is ON and that fuel is available before allowing acceleration.
- Arithmetic operators — used to calculate speed changes, distance ($\text{speed} \times \text{time}$) and fuel consumption.
- Boundary/limit checking — clamping speed to a 0–120 km/h range and fuel to a 0–50 litre range.
- try-except — used to catch invalid (non-numeric) input for acceleration, braking, driving time and refuelling.

IV. Program Working

- The car starts with speed 0 km/h, distance 0 km, fuel 20 litres, and the engine OFF.
- The engine must be started before the car can accelerate or drive; trying to stop the engine while the car is still moving is rejected.
- Accelerating increases the speed (capped at 120 km/h) and consumes a small amount of fuel; braking decreases the speed but never below 0.
- Driving calculates the distance travelled as $\text{Speed} \times \text{Time}$ and reduces the fuel proportionally to the distance — the journey is rejected if there isn't enough fuel.
- Refuelling adds fuel, capped at a 50-litre tank capacity, and the current status can be viewed at any time from the menu.

V. Algorithm / Steps

1. Create the car dictionary with speed = 0, distance = 0, fuel = 20 litres and engine = OFF.
2. Display the menu and take the user's choice.
3. Check whether the selected operation is currently allowed (e.g. engine ON, fuel available, car stopped).
4. Update the car's speed, distance, fuel or engine status according to the chosen operation.
5. Display the updated information after every action.
6. Repeat the menu until the user chooses Exit.

VI. Source Code

```
# ----- CAR DRIVE SIMULATION -----

def show_status(car):
    print("\n===== CAR STATUS =====")
    print("Speed:", car["speed"], "km/h")
    print("Distance:", car["distance"], "km")
    print("Fuel:", car["fuel"], "litres")
    print("Engine:", "ON" if car["engine"] else "OFF")

def start_engine(car):
    if car["engine"]:
        print("\nEngine is already ON.")
    else:
        car["engine"] = True
        print("\nEngine started successfully.")

def stop_engine(car):
    if car["speed"] > 0:
        print("\nCannot stop the engine while the car is moving.")
    elif not car["engine"]:
        print("\nEngine is already OFF.")
    else:
        car["engine"] = False
        print("\nEngine stopped.")

def accelerate(car):
    if not car["engine"]:
        print("\nStart the engine first.")
        return

    if car["fuel"] <= 0:
        print("\nNo fuel available.")
        return

    try:
        amount = float(input("Enter acceleration value (km/h): "))
```

```
    if amount <= 0:
        print("Acceleration must be positive.")
        return

    car["speed"] += amount

    if car["speed"] > 120:
        car["speed"] = 120
        print("Maximum speed reached: 120 km/h")

    car["fuel"] -= amount * 0.01

    if car["fuel"] < 0:
        car["fuel"] = 0

    print("Car accelerated successfully.")

except ValueError:
    print("Please enter a valid number.")

def brake(car):
    if car["speed"] == 0:
        print("\nCar is already stopped.")
        return

    try:
        amount = float(input("Enter braking value (km/h): "))

        if amount <= 0:
            print("Braking value must be positive.")
            return

        car["speed"] -= amount

        if car["speed"] < 0:
            car["speed"] = 0

        print("Car slowed down successfully.")

    except ValueError:
        print("Please enter a valid number.")

def drive(car):
    if not car["engine"]:
        print("\nStart the engine first.")
        return

    if car["speed"] <= 0:
```

```
        print("\nIncrease the speed first.")
        return

    try:
        time = float(input("Enter driving time (hours): "))

        if time <= 0:
            print("Time must be positive.")
            return

        distance = car["speed"] * time
        fuel_used = distance * 0.1

        if fuel_used > car["fuel"]:
            print("\nNot enough fuel for this journey.")
            return

        car["distance"] += distance
        car["fuel"] -= fuel_used

        print("\nCar drove successfully.")
        print("Distance travelled:", round(distance, 2), "km")

    except ValueError:
        print("Please enter a valid number.")

def refuel(car):
    try:
        amount = float(input("Enter fuel amount (litres): "))

        if amount <= 0:
            print("Fuel amount must be positive.")
            return

        car["fuel"] += amount

        if car["fuel"] > 50:
            car["fuel"] = 50
            print("Fuel tank is full. Maximum capacity is 50 litres.")

        print("Fuel added successfully.")
        print("Current fuel:", round(car["fuel"], 2), "litres")

    except ValueError:
        print("Please enter a valid number.")

def car_menu(car):
```

```
while True:
    print("\n=====")
    print("          CAR DRIVE SIMULATION")
    print("=====")
    print("1. Start Engine")
    print("2. Stop Engine")
    print("3. Accelerate")
    print("4. Brake")
    print("5. Drive")
    print("6. Refuel")
    print("7. Show Car Status")
    print("8. Exit")
    print("=====")

    choice = input("Enter your choice: ")

    if choice == "1":
        start_engine(car)

    elif choice == "2":
        stop_engine(car)

    elif choice == "3":
        accelerate(car)

    elif choice == "4":
        brake(car)

    elif choice == "5":
        drive(car)

    elif choice == "6":
        refuel(car)

    elif choice == "7":
        show_status(car)

    elif choice == "8":
        print("\nExiting Car Drive Simulation...")
        break

    else:
        print("\nInvalid choice. Please try again.")

def main():

    car = {
        "speed": 0,
        "distance": 0,
```

```
        "fuel": 20,  
        "engine": False  
    }  
  
    car_menu(car)  
  
main()
```

VII. Sample Input & Output

```
=====
CAR DRIVE SIMULATION
=====
1. Start Engine
2. Stop Engine
3. Accelerate
4. Brake
5. Drive
6. Refuel
7. Show Car Status
8. Exit
=====
Enter your choice: 2

Engine is already OFF.

=====
CAR DRIVE SIMULATION
=====
1. Start Engine
2. Stop Engine
3. Accelerate
4. Brake
5. Drive
6. Refuel
7. Show Car Status
8. Exit
=====
Enter your choice: 1

Engine started successfully.

=====
CAR DRIVE SIMULATION
=====
1. Start Engine
2. Stop Engine
3. Accelerate
```

VIII. Errors / Challenges Faced

- Preventing the speed from going negative when braking, and capping it at 120 km/h when accelerating.
- Calculating fuel consumption proportionally to both acceleration and the distance driven.
- Making sure the engine cannot be stopped while the car is still in motion.



PROJECT 3

Login System

Program File: *login_system.py*

I. Objective

To build a simple login system that creates a username and password, and then verifies them at login, allowing a maximum of three attempts.

II. Problem Statement

The program must not accept an empty username or a password shorter than 4 characters while creating an account, and must compare the entered username and password with the stored ones exactly, granting access only when both match.

III. Concepts Used

- Dictionary — the created username and password are stored together as one user dictionary.
- Functions with return values — `create_user()` returns the user dictionary, and `login()` returns True or False depending on whether login succeeded.
- while loop — used both to keep asking for a non-empty username / valid password while creating an account, and to allow up to 3 login attempts.
- if-else — used to compare the entered username and password with the stored values.
- String length validation — `len(password) < 4` is used to enforce a minimum password length.

IV. Program Working

- The user first creates an account by choosing a username (which cannot be empty) and a password (which must be at least 4 characters).
- The created username and password are stored in a dictionary, and the user is then asked to log in.
- At login, the entered username and password are compared with the stored values; both must match exactly for login to succeed.
- The user has 3 attempts; after 3 failed attempts, login access is blocked for that session.

V. Algorithm / Steps

1. Ask the user to create a username; keep asking if it is left empty.
2. Ask the user to create a password; keep asking until it is at least 4 characters long.
3. Store both values in a dictionary and set login attempts to 3.
4. Ask for the username and password and compare them with the stored values.
5. If both match, login is successful; otherwise, decrease the attempt count and try again.
6. Continue until login succeeds or the attempts run out, then block login for the current session.

VI. Source Code

```
# ----- LOGIN SYSTEM -----

def create_user():
    print("\n===== CREATE ACCOUNT =====")

    username = input("Create username: ").strip()

    while username == "":
        print("Username cannot be empty.")
        username = input("Create username: ").strip()

    while True:
        password = input("Create password: ")

        if len(password) < 4:
            print("Password must contain at least 4 characters.")
        else:
            break

    print("\nAccount created successfully!")
    return {
        "username": username,
        "password": password
    }

def login(user):
    print("\n===== LOGIN =====")

    attempts = 3

    while attempts > 0:
        username = input("Enter username: ")
        password = input("Enter password: ")

        if username == user["username"] and password == user["password"]:
            print("\nLogin successful!")
            print("Welcome,", user["username"])
            return True

        attempts -= 1

    if attempts > 0:
        print("\nInvalid username or password.")
        print("Attempts remaining:", attempts)

    print("\nToo many failed attempts.")
    print("Login access blocked for this session.")

    return False
```

```
def main():
    user = create_user()

    print("\nYour account is ready. Please login.")

    if login(user):
        print("\nYou are now logged into the system.")
    else:
        print("\nPlease try again later.")

main()
```

VII. Sample Input & Output

```
===== CREATE ACCOUNT =====
Create username: issaaa
Create password: ssaaai

Account created successfully!

Your account is ready. Please login.

===== LOGIN =====
Enter username: isha
Enter password: ahsi

Invalid username or password.
Attempts remaining: 2
Enter username: issaa
Enter password: aahsi

Invalid username or password.
Attempts remaining: 1
Enter username: ishaaa
Enter password: aassi

Too many failed attempts.
Login access blocked for this session.

Please try again later.
PS C:\Users\ASUS\OneDrive\Desktop\DSA-Python-Mini-Projects> |
```

VIII. Errors / Challenges Faced

- Making sure account creation rejects an empty username instead of silently accepting it.
- Comparing both the username and password together correctly, so a correct username with a wrong password still fails.

- Limiting login to exactly 3 attempts and blocking further tries afterwards.



PROJECT 4

Student Marks Analyzer

Program File: *student_marks_analyzer.py*

I. Objective

To take the marks of multiple students, calculate each student's grade and pass/fail status, and display the class average and the highest scorer.

II. Problem Statement

The program must validate that a student's name is not empty and that marks are entered between 0 and 100, assign a grade using nested if-elif-else conditions, store each student's data in a dictionary inside a list, and calculate the class average and the highest scorer by traversing that list.

III. Concepts Used

- List of dictionaries — every student is stored as a dictionary {name, marks, grade}, and all students are collected in one list.
- Functions with return values — `calculate_grade(marks)` returns the correct grade using if-elif-else.
- for loop with `range()` — used to take input for a user-defined number of students, and again to traverse the list for the average and highest scorer.
- Input validation — name cannot be empty; marks must be between 0 and 100 (re-asked in a while loop otherwise).
- Nested conditions — checking Pass/Fail first, then separately checking for a “Great performance” message when marks are 80 or above.
- Aggregate calculation — total and average marks are calculated by traversing the list and dividing by `len(students)`.
- Comparison / searching — the highest scorer is found by comparing every student's marks against the current highest, the same technique used in the Number Analyzer.

IV. Program Working

- The program asks how many students to enter, then collects each student's name and marks (0–100) one by one.
- Each student's grade is calculated using the marks (A+ for 90–100, down to F for below 50) and stored along with their name and marks.
- After all students are entered, the program displays every student's result, marking them Pass or Fail, with an extra “Great performance” message for marks of 80 or above.
- Finally, the class summary shows the average marks and identifies the highest scorer by comparing every student.

V. Algorithm / Steps

1. Ask for the number of students and create an empty student list.
2. For each student: take their name (must not be empty) and marks (must be between 0 and 100).

3. Calculate the grade using `calculate_grade()` and store `{name, marks, grade}` as a dictionary in the list.
4. Calculate total marks by traversing the list, then divide by the number of students to get the average.
5. Compare every student's marks to find the highest scorer.
6. Display every student's individual result, followed by the class summary (average marks and highest scorer).

VI. Source Code

```
# ----- STUDENT MARKS ANALYZER -----

def calculate_grade(marks):
    if marks >= 90:
        return "A+"
    elif marks >= 80:
        return "A"
    elif marks >= 70:
        return "B"
    elif marks >= 60:
        return "C"
    elif marks >= 50:
        return "D"
    else:
        return "F"

def main():

    students = []

    print("===== STUDENT MARKS ANALYZER =====")

    try:
        total_students = int(input("Enter number of students: "))

        if total_students <= 0:
            print("Number of students must be greater than 0.")
            return

        for i in range(total_students):

            print("\nStudent", i + 1)

            name = input("Enter student name: ").strip()

            while name == "":
                print("Name cannot be empty.")
                name = input("Enter student name: ").strip()

            marks = float(input("Enter marks (0-100): "))
```

```
while marks < 0 or marks > 100:
    print("Marks must be between 0 and 100.")
    marks = float(input("Enter marks (0-100): "))

grade = calculate_grade(marks)

student = {
    "name": name,
    "marks": marks,
    "grade": grade
}

students.append(student)

# Calculate average
total_marks = 0

for student in students:
    total_marks += student["marks"]

average = total_marks / len(students)

# Find highest scorer
highest = students[0]

for student in students:
    if student["marks"] > highest["marks"]:
        highest = student

# Display results
print("\n===== RESULT =====")

for student in students:
    print("\nName:", student["name"])
    print("Marks:", student["marks"])
    print("Grade:", student["grade"])

    if student["marks"] >= 50:
        print("Status: Pass")

        if student["marks"] >= 80:
            print("Keep it up! Great performance.")
    else:
        print("Status: Fail")

print("\n===== CLASS SUMMARY =====")
print("Average Marks:", round(average, 2))
print("Highest Scorer:", highest["name"])
print("Highest Marks:", highest["marks"])
```

```
except ValueError:  
    print("Please enter valid numbers.")  
  
main()
```

VII. Sample Input & Output



```
===== STUDENT MARKS ANALYZER =====
```

```
Enter number of students: 4
```

```
Student 1
```

```
Enter student name: isha
```

```
Enter marks (0-100): 98
```

```
Student 2
```

```
Enter student name: ahsi
```

```
Enter marks (0-100): 89
```

```
Student 3
```

```
Enter student name: naitik
```

```
Enter marks (0-100): 92
```

```
Student 4
```

```
Enter student name: risha
```

```
Enter marks (0-100): 99
```

```
===== RESULT =====
```

```
Name: isha
```

```
Marks: 98.0
```

```
Grade: A+
```

```
Status: Pass
```

```
Keep it up! Great performance.
```

```
Name: ahsi
```

```
Marks: 89.0
```

```
Grade: A
```

```
Status: Pass
```

```
Keep it up! Great performance.
```

```
Name: naitik
```

```
Marks: 92.0
```

```
Grade: A+
```

```
Status: Pass
```

```
Keep it up! Great performance.
```

```
===== RESULT =====  
  
Name: isha  
Marks: 98.0  
Grade: A+  
Status: Pass  
Keep it up! Great performance.  
  
Name: ahsi  
Marks: 89.0  
Grade: A  
Status: Pass  
Keep it up! Great performance.  
  
Name: naitik  
Marks: 92.0  
Grade: A+  
Status: Pass  
Keep it up! Great performance.  
  
Name: risha  
Marks: 99.0  
Grade: A+  
Status: Pass  
Keep it up! Great performance.  
  
===== CLASS SUMMARY =====  
Average Marks: 94.5  
Highest Scorer: risha  
Highest Marks: 99.0  
PS C:\Users\ASUS\OneDrive\Desktop\DSA-Python-Mini-Projects> |
```

VIII. Errors / Challenges Faced

- Re-prompting the user until marks fall within the valid 0–100 range, instead of accepting invalid values.
- Finding the highest scorer correctly by comparing every student, not just the first two.
- Keeping each student's name, marks and grade together correctly while looping through multiple students.

PROJECT 5

Number Analyzer

Program File: `number_analyzer.py`

I. Objective

To take a set of numbers from the user and analyse them by calculating their sum, subtraction, average, largest and smallest values, and the count of even and odd numbers.

II. Problem Statement

The program must accept a user-defined count of numbers, store them in a list, and calculate every result by traversing that list using functions — while handling invalid (non-numeric) input using exception handling.

III. Concepts Used

- Functions with parameters and return values — separate functions (`calculate_sum`, `calculate_subtraction`, `find_largest`, `find_smallest`) each take the numbers list as a parameter and return a result.
- Lists and `append()` — every number entered is added to a numbers list using `append()`.
- for loop (traversal) — used inside every function to visit each number one by one.
- Type conversion — `int()` for the count of numbers, `float()` for each number entered.
- Modulus operator — `number % 2 == 0` is used together with `is_integer()` to count even and odd whole numbers.
- try-except — catches `ValueError` if the user enters something that cannot be converted to a number.
- Searching — `find_largest()` and `find_smallest()` use the same start-with-the-first-value-and-compare technique described in the DSA concepts section.

IV. Program Working

- The program asks how many numbers the user wants to enter, then collects each one and stores it in a list.
- The sum is calculated by traversing the list and adding every number; the subtraction is calculated by subtracting every number after the first from it.
- The average is the sum divided by the number of values; the largest and smallest are found by comparing every number in the list.
- Every number is also checked with the modulus operator to count how many are even and how many are odd.

Note: `find_largest()` and `find_smallest()` in this program are a direct implementation of the linear search technique described in Section 8 — the exact same searching idea is reused again in the Student Marks Analyzer to find the class's highest scorer.

V. Algorithm / Steps

1. Ask how many numbers the user wants to enter and create an empty list.

2. Take each number from the user and append it to the list.
3. Calculate the sum and the subtraction by traversing the list.
4. Calculate the average as $\text{sum} \div \text{number of values}$.
5. Find the largest value by comparing every number against the current largest, and the smallest in the same way.
6. Check every number with the modulus operator and increase the even or odd counter accordingly.
7. Display all the results together.

VI. Source Code

```
def calculate_sum(numbers):
    total = 0

    for number in numbers:
        total += number

    return total

def calculate_subtraction(numbers):
    result = numbers[0]

    for number in numbers[1:]:
        result -= number

    return result

def find_largest(numbers):
    largest = numbers[0]

    for number in numbers:
        if number > largest:
            largest = number

    return largest

def find_smallest(numbers):
    smallest = numbers[0]

    for number in numbers:
        if number < smallest:
            smallest = number

    return smallest
```

```
def main():

    print("===== NUMBER ANALYZER =====")

    try:
        count = int(input("How many numbers do you want to enter? "))

        if count <= 0:
            print("Enter at least one number.")
            return

        numbers = []

        for i in range(count):
            number = float(input("Enter number " + str(i + 1) + ": "))
            numbers.append(number)

        total = calculate_sum(numbers)
        subtraction = calculate_subtraction(numbers)
        average = total / len(numbers)
        largest = find_largest(numbers)
        smallest = find_smallest(numbers)

        even_count = 0
        odd_count = 0

        for number in numbers:
            if number.is_integer() and int(number) % 2 == 0:
                even_count += 1
            elif number.is_integer():
                odd_count += 1

        print("\n===== RESULT =====")
        print("Numbers:", numbers)
        print("Sum:", total)
        print("Subtraction:", subtraction)
        print("Average:", round(average, 2))
        print("Largest Number:", largest)
        print("Smallest Number:", smallest)
        print("Even Numbers:", even_count)
        print("Odd Numbers:", odd_count)

    except ValueError:
        print("Please enter valid numbers.")

main()
```

VII. Sample Output

```
===== NUMBER ANALYZER =====
How many numbers do you want to enter? 5
Enter number 1: 10
Enter number 2: 25
Enter number 3: 15
Enter number 4: 40
Enter number 5: 20

===== RESULT =====
Numbers: [10.0, 25.0, 15.0, 40.0, 20.0]
Sum: 110.0
Subtraction: -90.0
Average: 22.0
Largest Number: 40.0
Smallest Number: 10.0
Even Numbers: 3
Odd Numbers: 2
```

VIII. Errors / Challenges Faced

- No output screenshot was captured for this program while running it, so the result shown below was produced by running the exact program above with the sample input 10, 25, 15, 40, 20.
- Handling text entered instead of a number using try-except so the program does not crash.
- Deciding to check `number.is_integer()` before classifying a value as even or odd, since the numbers are stored as floats.

SECTION 10

10. Tools and Technologies Used

Building and running five separate Python programs, one of which persists data to disk, involved a small but deliberate set of tools — each chosen for a specific purpose during the internship:

- Python 3 — the core programming language used to build all five mini projects.
- Visual Studio Code (VS Code) — used as the code editor for writing, indenting and running all the programs, with its integrated terminal used to test each menu interactively.
- Command Line / Terminal — used to run each program directly (e.g. `python atm_simulation.py`) and enter sample input interactively.
- json module — used in the ATM Simulation to store and retrieve account data from `atm_data.json` between runs.
- datetime module — used in the ATM Simulation to timestamp every deposit and withdrawal.
- Git and GitHub — used for version control and to host the DSA-Python-Mini-Projects repository, with the local `atm_data.json` test file excluded via `.gitignore` so real account data is never pushed to GitHub.



SECTION 11

11. Learning Outcomes

Through this internship and the five mini projects built during it, the following skills were practiced and strengthened — grouped here the same way the topics were introduced earlier in this report:

Python Fundamentals

- Variables, data types, input/output and type conversion.
- Operators, conditional statements (if-elif-else) and loops (for, while).
- Functions, parameters, arguments and return values.
- Lists, dictionaries, and lists of dictionaries.

Problem Solving

- Input validation, searching, traversal and counting.
- Breaking a larger problem down into smaller, single-purpose functions.
- Designing clear, menu-based program flow that a real user can follow.

File and Data Handling

- Reading and writing data using file handling and the json module.
- Working with dates and times using the datetime module.

Basic DSA Concepts

- Traversal, linear searching, and finding maximum/minimum values within a list.

Development Tools

- Writing and testing Python programs in VS Code.
- Using Git and GitHub to manage and share project code.

Overall takeaway: *The most valuable outcome of this internship was not any single topic, but seeing repeatedly — across five different projects — how the same small set of ideas (variables, conditions, loops, functions, lists, dictionaries) combine differently each time to solve a new problem.*

SECTION 12

12. Conclusion

This internship at UpEducative Technology Private Limited, under the guidance of mentor Md. Azad, provided a solid foundation in Python programming and basic Data Structures and Algorithms. Building five different mini projects — the ATM Simulation, Car Drive Simulation, Login System, Student Marks Analyzer and Number Analyzer — made it possible to see how core concepts such as variables, conditions, loops, functions, lists, dictionaries, file handling and exception handling come together to build complete, working programs rather than being used in isolation.

Each project also contributed something distinct: the ATM Simulation demonstrated persistent data storage with JSON; the Car Drive Simulation demonstrated state management through a single dictionary; the Login System demonstrated validation and controlled access; and the Student Marks Analyzer and Number Analyzer demonstrated the basic DSA techniques of traversal, searching and counting over a list. Together, they cover a genuinely broad cross-section of beginner-to-intermediate Python programming.

Overall, this internship was a valuable learning experience that improved both programming and problem-solving skills, and it has built a strong base for exploring more advanced Python topics, data structures and algorithms, and real-world application development in the future — including the possible improvements listed below.

Future Improvements

- Adding a graphical user interface (GUI) to the ATM Simulation and Car Drive Simulation.
- Storing Login System and Student Marks Analyzer data in a file or database, the way the ATM Simulation already does.
- Improving login and PIN security with hashing instead of storing plain values.
- Extending the Number Analyzer and Student Marks Analyzer with more DSA-based operations, such as sorting.

Student Declaration

I hereby declare that this internship report on “Python Programming”, submitted to Government Engineering College, Lakhisarai in partial fulfilment of the requirements for the degree of Bachelor of Technology in Computer Science and Engineering (Data Science), is a record of genuine work carried out by me during my internship at UpEducative Technology Private Limited under the supervision of Md. Azad.

Name: Isha Gupta

Registration No.: 25153158029

Branch: CSE (Data Science)

Session: 2026–2027

College: Government Engineering College, Lakhisarai

Signature: _____

Date: _____

